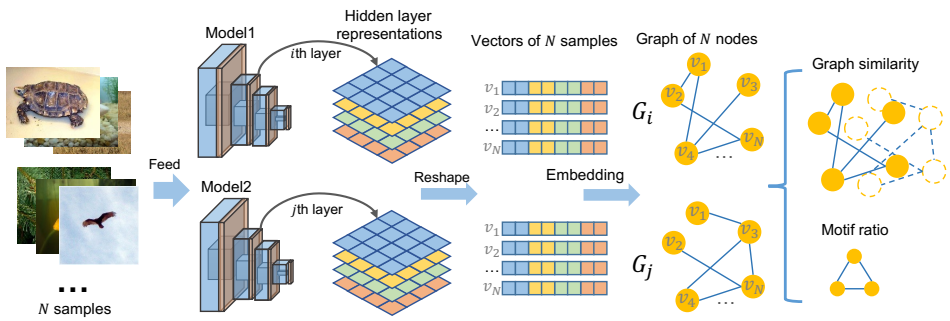


Figure 1: Pipeline of computing GBS.

3.1 CKA from Graph Perspective

As claimed by Kornblith et al [38], CKA first measures the similarity between every pair of examples in their representation separately, and then compares the similarity structure. This process can be understood as the construction and comparison of graphs, which is intuitive with the linear kernel. Given two representations $\mathbf{X}$ and $\mathbf{Y}$, the linear kernel Hilbert-Schmidt Independence Criterion [29] (HSIC) is

$$HSIC(K_X, K_Y) = \frac{1}{(N-1)^2} tr(K_X \mathbf{J} K_Y \mathbf{J}) = \frac{1}{(N-1)^2} tr(\mathbf{X}\mathbf{X}^T \mathbf{J} \mathbf{Y}\mathbf{Y}^T \mathbf{J}), \tag{1}$$

where N is the number of examples, $J = I_N - \frac{1}{N}\mathbf{1}$ is the centering matrix, and $K_X(\mathbf{X}, \mathbf{X})$ is the kernel. Representations $\mathbf{X} \in \mathbb{R}^{N \times D}$ and $\mathbf{Y} \in \mathbb{R}^{N \times D}$ are both obtained by N examples, where D is feature dimensions.

Considering the physical meaning of $\mathbf{X}$ and $\mathbf{Y}$ that each row is a layer's output for different examples, linear HSIC embeds representations $\mathbf{X}$ and $\mathbf{Y}$ into two adjacent matrices $\mathbf{A}_X = \mathbf{X}\mathbf{X}^T$ and $\mathbf{A}_Y = \mathbf{Y}\mathbf{Y}^T$, where nodes are input examples and edges are the inner product of the examples outputs in the same layer. $\mathbf{A}_X$ and $\mathbf{A}_Y$ describe the relationships between input examples and the role of the centering matrix can be regarded as subtracting a constant from each item of the adjacent matrices, thus does not change the essence of its physical meaning. Assuming centered adjacent matrices are $\tilde{\mathbf{A}}_X$ and $\tilde{\mathbf{A}}_Y$, that both of them are weighted undirected graph, i.e., $\tilde{\mathbf{A}}_X$ and $\tilde{\mathbf{A}}_Y$ are symmetric. The diagonal of $\tilde{\mathbf{A}}_X \tilde{\mathbf{A}}_Y$ represents the inner product of first-order neighbor information between each node. HSIC then averages the elements in the diagonal as the final result. CKA further normalizes HSIC by

$$CKA(\mathbf{X}, \mathbf{Y}) = \frac{HSIC(K_X, K_Y)}{\sqrt{HSIC(K_X, K_X) HSIC(K_Y, K_Y)}}, \tag{2}$$

because HSIC is a test statistic for independence instead of mutual information and normalization can induce invariance to isotropic scaling.

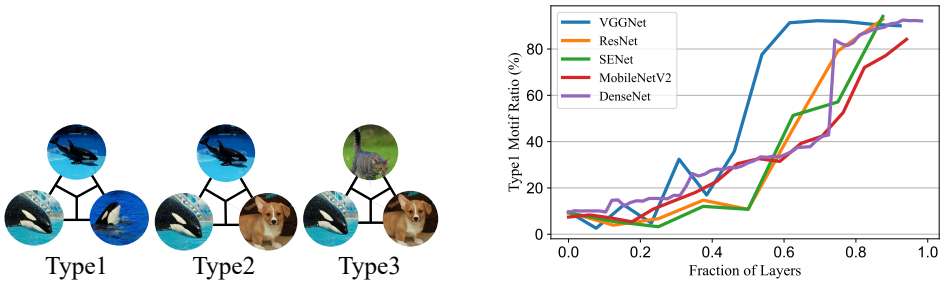
Nevertheless, we argue that there is still room for improvement in graph construction and similarity comparison. The topology of the kernel-constructed graph is fully connected, which may bring noise for similarity measurement. Hence, pruning irrelevant edges may improve the performance. The linear HSIC with sparse connections is

$$\frac{1}{(m-1)^2} tr(M(\mathbf{A}_X, m) \mathbf{J}_m M(\mathbf{A}_Y, m) \mathbf{J}_m), \tag{3}$$

where m is the number of reserved edges, $M(\cdot, \cdot)$ represents reserving top m edges in each row of the adjacent matrix, and $\mathbf{J}_m = I_N - \frac{1}{m}\mathbf{1}$. Meanwhile, considering that rows and columns of representations have clear physical meaning, the properties of the graph can reveal more information to us.

3.2 Graph-Based Similarity

As shown in Fig. 1, GBS requires a batch of samples to obtain the hidden layer representations. After that, representations of a layer are embedded in a graph, then we compare the similarity between graphs.



(a) Motif types.

(b) Variation of type1 motif ratio with layer depth.

Figure 2: Motifs in neural networks.

Given a set of samples $\mathcal{X} = \{x_1, x_2, \ldots, x_N\}$ where N represents the number of samples derived from the dataset, we feed them into the target model with fixed parameters to obtain neural network hidden representations $\mathbf{F}_i \in \mathbb{R}^{N \times C_i \times W_i \times H_i}$, $i = 1, \cdots, L$, where L is the number of layers, W_i and H_i are width and height of feature maps, and C_i is the number of channels in ith layer, respectively. Note that we treat a building block containing multiple convolutional layers as a layer for ResNet [33] and treat a single convolutional layer as a layer for VGGNet [60]. Then we reshape the obtained representation to reduce its dimension, i.e., $\mathbb{R}^{N \times C_i \times W_i \times H_i} \rightarrow \mathbb{R}^{N \times M_i}$, where $M_i = C_i \times W_i \times H_i$.

We define the graph of ith layer corresponding to the input $\mathcal{X}$ as $\mathcal{G}_i = (\mathcal{V}, \mathcal{E})$ by a node set $\mathcal{V} = \{v_1, ..., v_N\}$ and an edge set $\mathcal{E} \subseteq \{(v_n, v_m) | v_n, v_m \in \mathcal{V}\}$. The number of nodes in $\mathcal{G}_i$ is determined by the number of samples N, which is independent of the feature map size. Each node v_k corresponds to its input x_k and the feature of v_k is $\mathbf{F}_{i|k}$. The edge weight between v_n and v_m is calculated by the cosine similarity of their features

$$e_{nm} = \frac{\mathbf{F}_{i|n} \cdot \mathbf{F}_{i|m}^T}{\|\mathbf{F}_{i|n}\|_2 \cdot \|\mathbf{F}_{i|m}\|_2}. \tag{4}$$

Compared with linear CKA, we normalized the edge weight, only considering the angle between representations. Because the amplitude of DNNs representations does not determine their performance, e.g., a binary neural network [52] can also obtain high accuracy on various tasks.

Then we obtain the similarity of layer i and layer j by computing the similarity s_{ij} between $\mathcal{G}_i$ and $\mathcal{G}_j$. We use Layer Similarity [80] (LSim) to calculate the similarity between layers. LSim is proposed to measure the similarity between layers of complex and multiplex networks. Assuming the adjacent matrices of $\mathcal{G}_i$ and $\mathcal{G}_j$ are $\mathbf{A}_i$ and $\mathbf{A}_j$, the similarity is calculated by

$$s_{ij} = \frac{1}{N} \sum_{k=1}^{N} f_{cos}(\mathbf{a}_{i,k}, \mathbf{a}_{j,k}), \tag{5}$$

where $\mathbf{a}_{i,k}$ is the row vector of $\mathbf{A}_i$ and $f_{cos}(\cdot)$ is the cosine similarity between two vectors.

However, when N is large, a fully connected graph will induce noise in the result, since many irrelevant nodes are also connected. To obtain a better result, we only connect the top k nodes (k equals the degree of the node) with the highest similarity at the step of graph construction.

As introduced in Section 2, a good similarity index should be invariant to orthogonal transformation and isotropic scaling but not invertible linear transform. GBS satisfies the requirements and we provide the proof in the appendix.

3.3 Motif and Feature Clustering

In addition to similarity measurement, we use the triangle motifs [70] as the functional similarity index. Triangle motif consists of three nodes that are all fully connected.

As shown in Fig. 2 (a), there are three types of motifs according to their ground truth labels. Since previous works [12, 69, 22] indicate that features of the same category tend to cluster in deep layers, we focus on the ratio of the first type (type1) across all the motifs in a graph, i.e., samples with the same labels are connected.

Algorithm 1: Breadth-First Motif Search

input : A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and graph size N (number of nodes).
output : Number of different types of motifs.

```
for i=1 to N do
    Initialize a motif queue Q with an incomplete motif q_0 = {v_i, , };
    while Q is not empty do
        q = dequeue(Q);
        if q has 3 neighbors then
            Add one to the number of motifs of the corresponding type;
            continue
        end
        get v_current, which is the last node in q; // e.g., when q = {v_1, , }, v_current = v_1
        for j=current to N do
            if v_j has been visited then
                continue
            end
            if v_current is connected to v_j then
                q_new = copy(q);
                add v_j to q_new;
                enqueue(q_new, Q);
            end
        end
    end
end
```

As shown in Alg. 1, we design a Breadth-First Search (BFS) based strategy to count the number of different types of triangle motifs. Specifically, we will traverse all nodes in the graph as the root node of searching. For each root node, we initialize a queue $\mathcal{Q}$ with an element $q_0 = \{v_i, \ , \ \}$ that only has a root node. When $\mathcal{Q}$ is not empty, we do the following loop: **Step 1.** We dequeue $\mathcal{Q}$ to obtain q and count the number of nodes in q. If the number of nodes is equal to 3, we first verify whether these nodes form a triangle motif. If they do, we classify the triangle motif according to the label of each node and execute the next loop, otherwise, execute step 2. **Step 2.** For the last node $v_{current}$ in q, we traverse all nodes whose node number is greater than $v_{current}$. If the node is connected with $v_{current}$ and has not been visited, we add it to q and enqueue q to $\mathcal{Q}$. According to the above steps, we can count the number of three types of triangle motifs and calculate the proportion of each type of triangle motif.

Fig. 2 (b) shows the type1 motif ratio as a function of model depth in different models. It demonstrates that the ratio of type1 motif increases with the depth. Hence, we argue that the ratio of the type1 motif is an indicator of whether useful features have been extracted.

4 Experiments

4.1 Experimental Settings

We conduct our experiments on CIFAR10, CIFAR100 [39], and ImageNet [17]. The three datasets contain 10, 100, and 1000 categories of images, respectively. The images size of CIFAR10 and CIFAR100 is 32×32 while that of ImageNet is 224×224. The networks we used are 10layer-CNN [62], VGGNet [60], and ResNet [33]. Their detailed architectures are provided in the appendix.

4.2 Ablation Analysis and Sanity Check

Our ablation analysis focus on the influence of 1) the number of samples (graph size) and 2) the number of connected nodes (degree of the graph). We use the sanity check to evaluate the performance of GBS as described in Section 2. We use ten ResNet18, VGGNet16, and 10layer-CNN of different initialization trained on CIFAR10, and compute the average accuracy over all the layers. The